

Agent Skills

Agent Skills are modular capabilities that extend Claude's functionality. Each Skill packages instructions, metadata, and optional resources such as scripts and templates that Claude uses automatically when relevant.

Note

This feature is **not** eligible for [Zero Data Retention \(ZDR\)](#). Data is retained according to the feature's standard retention policy.

Why Use Skills?

Skills are reusable, filesystem-based resources that give Claude domain-specific expertise: workflows, context, and best practices that turn a general-purpose agent into a specialist.

Unlike prompts, which provide conversation-level instructions for one-off tasks, Skills load on demand. You do not have to repeat the same guidance across conversations.

Key Benefits

- **Specialize Claude:** Tailor capabilities for domain-specific tasks.
- **Reduce repetition:** Create guidance once and use it automatically.
- **Compose capabilities:** Combine multiple Skills for complex, multi-step tasks.

Note

For more on the architecture and real-world applications of Agent Skills, see:

[Equipping agents for the real world with Agent Skills](#)

Using Skills

Anthropic provides pre-built Agent Skills for common document tasks, including:

- PowerPoint
- Excel
- Word
- PDF

You can also create your own custom Skills.

Both work the same way: once a Skill is available in your environment, Claude uses it automatically when relevant to your request.

Pre-Built Agent Skills

Pre-built Agent Skills are available on:

- [claude.ai](#)
- [Claude API](#)
- [Claude Platform on AWS](#)
- [Microsoft Foundry](#)

On Microsoft Foundry, Agent Skills require a [Hosted on Anthropic deployment](#).

See [Available Skills](#) for the complete list.

Custom Skills

Custom Skills let you package:

- Domain expertise
- Organizational knowledge
- Repeatable workflows
- Reference material
- Executable scripts
- Templates and examples

They are available across Claude's products:

- Create them directly in Claude Code.
- Upload them through the Claude API.
- Add them through [claude.ai](#) settings.
- Upload them through the Skills API on Claude Platform on AWS and Microsoft Foundry.

Get Started

- **Pre-built Agent Skills:** See the [quickstart tutorial](#) to start using PowerPoint, Excel, Word, and PDF Skills in the API.
- **Custom Skills:** See the [Agent Skills Cookbook](#).

How Skills Work

Skills use Claude's virtual-machine environment to provide capabilities beyond what is possible with prompts alone.

Claude operates in a virtual machine with:

- Filesystem access
- Bash commands
- Code execution capabilities

Skills exist as directories containing instructions, executable code, and reference materials. Think of them like an onboarding package you would create for a new team member.

This filesystem-based architecture enables **progressive disclosure**: Claude loads information in stages as needed instead of consuming all of it upfront.

Level 1: Metadata — Always Loaded

The Skill's YAML frontmatter provides discovery information.

```
---
name: pdf-processing
description: Extract text and tables from PDF files, fill forms, merge
documents. Use when working with PDF files or when the user mentions PDFs,
forms, or document extraction.
---
```

Claude loads this metadata at startup and includes it in the system prompt.

The `description` determines when Claude should trigger the Skill. It should explain:

1. What the Skill does.
2. When Claude should use it.

Because only the name and description are loaded initially, you can install many Skills without a large context penalty.

Level 2: Instructions — Loaded When Triggered

The main body of `SKILL.md` contains procedural knowledge such as:

- Workflows
- Best practices
- Guidance
- Decision rules

Example:

```
# PDF Processing

## Quick Start

Use pdfplumber to extract text from PDFs:

```python
import pdfplumber

with pdfplumber.open("document.pdf") as pdf:
 text = pdf.pages[0].extract_text()
...

For advanced form filling, see FORMS.md.
```

When a request matches the Skill's description, Claude reads `SKILL.md` from the filesystem.

Only then does the full Skill content enter the context window.

---

## Level 3+: Resources and Code — Loaded as Needed

Skills can bundle additional materials:

```
pdf-processing/
├── SKILL.md
├── FORMS.md
├── REFERENCE.md
└── scripts/
 └── fill_form.py
```

### Instructions

Additional Markdown files such as:

- `FORMS.md`
- `REFERENCE.md`

These contain specialized guidance and workflows.

### Code

Executable scripts such as:

```
fill_form.py
validate.py
```

Claude runs these scripts through Bash.

The important efficiency advantage is that the script's source code does not need to enter Claude's context. Only the script's output is returned.

## Resources

Reference materials can include:

- Database schemas
- API documentation
- Templates
- Examples
- Large datasets

Claude accesses these files only when needed.

---

## Progressive Disclosure Summary

Level	When Loaded	Token Cost	Content
<b>Level 1: Metadata</b>	Always, at startup	~100 tokens per Skill	<code>name</code> and <code>description</code> from YAML frontmatter
<b>Level 2: Instructions</b>	When the Skill is triggered	Under 5k tokens	<code>SKILL.md</code> body with instructions and guidance
<b>Level 3+: Resources</b>	As needed	None until accessed	Bundled files, references, and scripts

Progressive disclosure ensures that only relevant content occupies the context window.

---

## The Skills Architecture

Skills run in a code-execution environment where Claude has access to:

- The filesystem
- Bash
- Code execution

Skills exist as directories on a virtual machine, and Claude interacts with them using normal filesystem and shell commands.

## Agent Skills Architecture

### How Claude Accesses Skill Content

When a Skill is triggered:

1. Claude uses Bash to read `SKILL.md`.
  2. The instructions enter the context window.
  3. If `SKILL.md` references other files, Claude reads those files as needed.
  4. If the Skill references executable scripts, Claude runs them.
  5. Only script output is returned to Claude's context.
- 

### What This Architecture Enables

#### On-Demand File Access

Claude reads only the files needed for a particular task.

A Skill may contain dozens of reference files, but if the task only requires the sales schema, Claude loads only that schema.

Everything else remains on the filesystem and consumes zero context tokens.

#### Efficient Script Execution

When Claude runs:

```
python validate_form.py
```

the script's source code does not enter the context window.

Only its output does:

```
Validation passed
```

or:

```
Error: Required field "account_number" is missing.
```

This can be far more efficient than having Claude generate equivalent code dynamically.

## No Practical Limit on Bundled Content

Files do not consume context until accessed.

A Skill can therefore include:

- Comprehensive API documentation
- Large reference collections
- Extensive examples
- Large datasets
- Complex scripts

without paying an upfront context cost.

---

## Example: Loading a PDF-Processing Skill

Suppose you created a custom Skill named `pdf-processing`.

### 1. Startup

Claude's system prompt includes:

```
pdf-processing - Extract text and tables from PDF files, fill forms, merge documents. Use when working with PDF files or when the user mentions PDFs, forms, or document extraction.
```

### 2. User Request

```
Extract the text from this PDF and summarize it.
```

### 3. Claude Invokes the Skill

```
cat pdf-processing/SKILL.md
```

The Skill's instructions are loaded into context.

## 4. Claude Determines What Else Is Needed

The request does not involve form filling, so Claude does not read:

```
FORMS.md
```

## 5. Claude Executes the Task

Claude follows the instructions in `SKILL.md` and completes the PDF-processing workflow.

Skills Loading into Context Window

---

# Where Skills Work

Skills are available across Claude's agent products.

### Note

Claude Platform on AWS and Microsoft Foundry inherit the same Skills behavior as the Claude API unless otherwise noted.

---

## Claude API

The Claude API supports:

- Pre-built Agent Skills
- Custom Skills

Specify the relevant `skill_id` in the `container` parameter along with the [code execution tool](#).

## Prerequisites

Using Skills through the API requires:

1. The [code execution tool](#).
2. The Skills beta header:

```
skills-2025-10-02
```

When using the [Files API](#) to upload inputs or download Skill-generated files, also include:



```
files-api-2025-04-14
```

## Pre-Built Skill IDs

Use:

- pptx
- xlsx
- docx
- pdf

## Custom Skills

Create and upload custom Skills through the Skills API:

```
/v1/skills
```

Custom Skills uploaded through the API are shared workspace-wide.

## API Runtime Constraints

Skills on the API run in a sandboxed container with:

- No network access
- No runtime package installation

See [Limitations and Constraints](#).

For more information:

[Using Agent Skills with the API](#)

---

## Claude Code

[Claude Code](#) supports custom Skills.

The pre-built document Skills for:

- PowerPoint
- Excel
- Word
- PDF

are not available directly in Claude Code.

The open-source [Claude API Skill](#) is bundled with Claude Code.

## Skill Locations

Personal Skills:

```
~/ .claude/skills/
```

Project Skills:

```
.claude/skills/
```

Claude discovers and uses these Skills automatically.

For more information:

[Use Skills in Claude Code](#)

---

## claude.ai

[claude.ai](#) supports:

- Pre-built Agent Skills
- Custom Skills

### Pre-Built Skills

These are automatically active when you create documents.

### Custom Skills

Upload your own Skills as ZIP files through:

```
Settings > Features
```

Custom Skills are available on:

- Pro
- Max
- Team
- Enterprise

Code execution must be enabled.

Custom Skills on claude.ai are individual to each user. They are not automatically shared organization-wide.

## Related Help Center Resources

- [What are Skills?](#)
- [Using Skills in Claude](#)
- [How to create custom Skills](#)
- [Teach Claude your way of working using Skills](#)

---

# Skill Structure

Every Skill requires a `SKILL.md` file with YAML frontmatter.

```

name: your-skill-name
description: Brief description of what this Skill does and when to use it

Your Skill Name

Instructions

[Clear, step-by-step guidance for Claude to follow]

Examples

[Concrete examples of using this Skill]
```

## Required Fields

- `name`
- `description`

### `name` Requirements

The `name` :

- Must be no more than 64 characters.
- May contain only lowercase letters, numbers, and hyphens.
- Cannot contain XML tags.
- Cannot contain the reserved words:

- anthropic
- claude

## description Requirements

The description :

- Must not be empty.
- Must be no more than 1,024 characters.
- Cannot contain XML tags.
- Must explain:
  - What the Skill does.
  - When Claude should use it.

For complete authoring guidance:

[Skill Authoring Best Practices](#)

---

## Security Considerations

Use Skills only from trusted sources:

- Skills you created yourself.
- Skills obtained directly from Anthropic.
- Skills you have thoroughly audited.

Skills give Claude new capabilities through instructions and executable code.

A malicious Skill could direct Claude to:

- Invoke tools inappropriately.
- Execute harmful commands.
- Access sensitive files.
- Exfiltrate data.
- Perform unauthorized system operations.

### Warning

If you must use a Skill from an untrusted or unknown source, exercise extreme caution and thoroughly audit it before use.

Depending on the access Claude has while executing the Skill, malicious Skills could lead to:

- Data exfiltration
- Unauthorized system access
- Other security risks

# Key Security Practices

## Audit Thoroughly

Review every bundled file:

- SKILL .md
- Scripts
- Images
- Reference files
- Templates
- Configuration

Look for suspicious behavior such as:

- Unexpected network calls
- Unusual filesystem access
- Commands unrelated to the Skill's stated purpose

## Treat External Sources as Risky

Skills that fetch content from external URLs pose additional risks.

Fetches content may contain malicious instructions.

Even trustworthy Skills can become dangerous if their external dependencies are compromised or change unexpectedly.

## Watch for Tool Misuse

A malicious Skill may attempt to invoke:

- Bash
- File operations
- Code execution
- External APIs
- Other tools

in ways that do not match its stated purpose.

## Protect Sensitive Data

Skills with access to sensitive information could intentionally or accidentally expose that data.

## Treat Skills Like Software

Installing a Skill should be treated similarly to installing software, especially in environments with access to:

- Production systems
- Sensitive data
- Credentials
- Critical infrastructure

For organization-scale governance:

[Skills for Enterprise](#)

---

# Available Skills

## Pre-Built Agent Skills

The following pre-built Skills are available:

### PowerPoint — pptx

Capabilities include:

- Create presentations
- Edit slides
- Analyze presentation content

### Excel — xlsx

Capabilities include:

- Create spreadsheets
- Analyze data
- Generate reports
- Create charts

### Word — docx

Capabilities include:

- Create documents
- Edit content
- Format text

## PDF — pdf

Capabilities include:

- Generate formatted PDF documents
- Create reports

These Skills are available on:

- Claude API
- Claude Platform on AWS
- Microsoft Foundry
- claude.ai

See the [quickstart tutorial](#).

---

## Open-Source Skills

Anthropic publishes open-source Skills in the:

[Anthropic Skills Repository](#)

### Claude API Skill

The [Claude API Skill](#) provides:

- Up-to-date API reference material
- SDK documentation
- Best practices
- Support for eight programming languages

It is bundled with Claude Code and is also available from the Skills repository.

---

## Custom Skill Examples

For complete custom Skill examples:

[Skills Cookbook](#)

---

## Data Retention

Agent Skills are not covered by Zero Data Retention arrangements.

Skill definitions and execution data are retained according to Anthropic's standard data-retention policy.

For ZDR eligibility across features:

[API and Data Retention](#)

---

# Limitations and Constraints

Claude Platform on AWS and Microsoft Foundry follow the same limitations as the Claude API unless otherwise stated.

---

## Cross-Surface Availability

**Custom Skills do not automatically sync across surfaces.**

### claude.ai → API

Skills uploaded to claude.ai must be separately uploaded to the API.

### API → claude.ai

Skills uploaded through the API are not automatically available on claude.ai.

## Claude Code

Claude Code Skills are filesystem-based and separate from both:

- claude.ai
- Claude API

You must manage Skills separately for each surface.

---

## Sharing Scope

Surface	Sharing Model
<b>claude.ai</b>	Individual user only
<b>Claude API</b>	Workspace-wide
<b>Claude Code</b>	Personal or project-based



## claude.ai

Each team member must upload custom Skills separately.

There is no centralized organization-wide administration for custom Skills.

## Claude API

Uploaded Skills are available workspace-wide.

## Claude Code

Skills can be:

Personal:

```
~/ .claude/skills/
```

Project-based:

```
.claude/skills/
```

They can also be shared through Claude Code Plugins.

---

# Runtime Environment Constraints

The exact runtime environment depends on the Claude product.

## claude.ai

Network access may be:

- Full
- Partial
- Disabled

depending on user and administrator settings.

See:

[Create and Edit Files](#)

---

## Claude API

### No Network Access

Skills cannot:

- Call external APIs
- Access the internet

### No Runtime Package Installation

You cannot install new packages during execution.

### Preconfigured Dependencies Only

Only preinstalled packages are available.

See the [Code Execution Tool](#) documentation for the available package list.

---

## Claude Code

### Full Network Access

Skills have the same network access as other programs running on your computer.

### Global Package Installation Is Discouraged

Skills should install dependencies locally whenever possible to avoid interfering with the user's system.

---

## Next Steps

### Get Started with Agent Skills in the API

Learn how to use Agent Skills to create documents with the Claude API:

[Quickstart Tutorial](#)

### Using Agent Skills with the API

Learn how to extend Claude's capabilities through the API:

[Skills Guide](#)

## Use Skills in Claude Code

Create and manage custom Skills:

[Claude Code Skills](#)

## Skill Authoring Best Practices

Learn how to write Skills that Claude can reliably discover and use:

[Skill Authoring Best Practices](#)

---

## Documentation Index

Fetch the complete documentation index at:

```
https://code.claude.com/docs/llms.txt
```

Use this file to discover all available pages before exploring further.

---

## Extend Claude with Skills

Skills extend what Claude can do in Claude Code.

Create a `SKILL.md` file containing instructions, and Claude adds the Skill to its toolkit.

Claude can:

- Automatically use a Skill when relevant.
- Let you invoke a Skill directly with:

```
/skill-name
```

Create a Skill when you repeatedly paste the same:

- Instructions
- Checklist
- Multi-step procedure
- Workflow

A Skill is also a good replacement when part of `CLAUDE.md` has grown into a procedure rather than a persistent project fact.

Unlike `CLAUDE.md`, a Skill's full body loads only when the Skill is used. This means long reference material costs almost nothing until it is actually needed.

### Note

For built-in commands such as `/help` and `/compact`, and bundled Skills such as `/debug` and `/code-review`, see the [commands reference](#).

### Custom commands have been merged into Skills.

These both create `/deploy`:

```
.claude/commands/deploy.md
```

```
.claude/skills/deploy/SKILL.md
```

Existing `.claude/commands/` files continue to work.

Skills add additional capabilities:

- Supporting files
- Invocation controls
- Automatic activation
- Subagent execution
- Dynamic context injection

Claude Code Skills follow the [Agent Skills open standard](#).

Claude Code extends the standard with features including:

- [Invocation control](#)
- [Subagent execution](#)
- [Dynamic context injection](#)

---

## Bundled Skills

Claude Code includes bundled Skills available in every session unless disabled using:

```
disableBundledSkills
```

Examples include:

- `/doctor`
- `/code-review`
- `/batch`
- `/debug`
- `/loop`
- `/claude-api`

Unlike most built-in commands, which execute fixed logic directly, bundled Skills are prompt-based.

They provide Claude with detailed instructions and let Claude orchestrate the work using its tools.

Invoke them just like any other Skill:

```
/skill-name
```

The `/doctor` setup checkup is an exception.

In newer Claude Code versions, `/doctor` remains available even when bundled Skills are disabled.

To hide it, use either:

```
DISABLE_DOCTOR_COMMAND
```

or a `skillOverrides` entry:

```
{
 "skillOverrides": {
 "doctor": "off"
 }
}
```

Bundled Skills appear alongside built-in commands in the [commands reference](#) and are marked **Skill** in the Purpose column.

---

# Run and Verify Your App

Three bundled Skills work together to launch an application and verify changes against the actual running application.

Skill	Purpose
<code>/run</code>	Launch and drive your application to see a change working
<code>/verify</code>	Build and run the application to confirm a change works without falling back to tests or type checks
<code>/run-skill-generator</code>	Teach <code>/run</code> and <code>/verify</code> how to build and launch your project

These Skills can infer how to launch common project types from files such as:

- `README`
- `package.json`
- `Makefile`

However, inference becomes less reliable when a project requires:

- A database
- Environment files
- A graphical session
- Multi-step builds
- Other nonstandard setup

## `/run-skill-generator`

The `/run-skill-generator` Skill records the successful application-launch recipe.

It captures:

- Installation commands
- Environment variables
- Launch scripts
- Other setup requirements

It creates a project Skill such as:

```
.claude/skills/run-<name>/
```

After that:

- `/run`
- `/verify`
- Other agents working in the repository

can follow the recorded recipe instead of rediscovering the process.

Run `/run-skill-generator`:

- Once per project.
- Again whenever the build or launch process changes.

---

## Getting Started

### Create Your First Skill

This example creates a Skill that:

- Summarizes uncommitted Git changes.
- Flags potentially risky changes.

It injects the live Git diff before Claude reads the Skill, grounding the response in the current working tree.

Claude can invoke it automatically when you ask about your changes, or you can invoke it directly with:

```
/summarize-changes
```

---

### Step 1: Create the Skill Directory

Create a personal Skill directory:

```
mkdir -p ~/.claude/skills/summarize-changes
```

Personal Skills are available across all your projects.

---

### Step 2: Create `SKILL.md`

Save the following file as:

```
~/.claude/skills/summarize-changes/SKILL.md
```

```

description: Summarizes uncommitted changes and flags anything risky. Use when
the user asks what changed, wants a commit message, or asks to review their
diff.

```

### ## Current Changes

```
!`git diff HEAD`
```

### ## Instructions

Summarize the changes above in two or three bullet points, then list any risks you notice, such as:

- Missing error handling
- Hardcoded values
- Tests that need updating

If the diff is empty, say there are no uncommitted changes.

The line:

```
!`git diff HEAD`
```

uses **dynamic context injection**.

Claude Code runs the command before Claude sees the Skill content.

The command output replaces the line, so Claude receives the actual current Git diff directly inside the Skill instructions.

---

## Step 3: Test the Skill

Open a Git project.

Make a small edit.

Start Claude Code:



claude

Automatic Invocation

Ask:

What did I change?

Claude should recognize that the request matches the Skill description and load it automatically.

Direct Invocation

Run:

/summarize-changes

Claude should return:

- A short summary of your changes.
- A list of risks.

# Where Skills Live

Where you store a Skill determines who can use it.

Location	Path	Applies To
Enterprise	Managed settings	All users in the organization
Personal	~/ .claude/skills/<skill-name>/SKILL.md	All your projects
Project	.claude/skills/<skill-name>/SKILL.md	Current project
Plugin	<plugin>/skills/<skill-name>/SKILL.md	Wherever the plugin is enabled

Skill Precedence

When Skills share the same name:

```
Enterprise > Personal > Project
```

A Skill at any of these levels can also override a bundled Skill.

For example:

```
.claude/skills/code-review/
```

can replace the bundled:

```
/code-review
```

Plugin Skills use a namespace:

```
/plugin-name:skill-name
```

so they do not conflict with Skills at other levels.

---

## Commands and Skills with the Same Name

Files in:

```
.claude/commands/
```

continue to work.

However, when both exist:

```
.claude/commands/deploy.md
```

and:

```
.claude/skills/deploy/SKILL.md
```

the Skill takes precedence.

---

# Nested Skills

Claude Code can load Skills from nested `.claude/skills/` directories below the working directory.

This is particularly useful in monorepositories.

Example:

```
repo/
├── .claude/
│ └── skills/
│ └── deploy/
│ └── SKILL.md
└── apps/
 └── web/
 └── .claude/
 └── skills/
 └── deploy/
 └── SKILL.md
```

The project-root Skill is:

```
/deploy
```

The nested Skill is available as:

```
/apps/web:deploy
```

Its description indicates which directory it applies to.

Typing:

```
/deploy
```

runs the project-root Skill.

Typing:

```
/apps/web:deploy
```

runs the nested version explicitly.

When the unqualified Skill is invoked, Claude Code can also identify relevant nested variants and invoke those that apply to the files being worked on.

---

## Symlinked Skills

A Skill directory in:

```
~/ .claude/skills/
```

or:

```
.claude/skills/
```

can be a symbolic link to another directory.

Claude Code follows the symbolic link and reads:

```
SKILL.md
```

from the target.

If the same Skill directory is reachable through multiple locations, Claude Code loads it only once.

Plugin Skills handle symlinks differently.

See:

[Share files within a marketplace with symlinks](#)

---

## Turn a Skill Folder into a Plugin

Add:

```
.claude-plugin/plugin.json
```

to a Skill folder.

The directory can then load as a plugin named:

```
<name>@skills-dir
```

A plugin can bundle:

- Skills
- Agents
- Hooks
- MCP servers
- Output styles

For project Skills, you must first accept the workspace trust dialog.

---

## Live Change Detection

Claude Code watches Skill directories for changes.

Changes to:

```
SKILL.md
```

inside:

- `~/.claude/skills/`
- Project `.claude/skills/`
- `.claude/skills/` directories added with `--add-dir`

can take effect during the current session without restarting Claude Code.

If the top-level Skills directory did not exist when the session started, creating it may require restarting Claude Code before it can be watched.

### Note

Live change detection covers `SKILL.md` text.

For a Skill folder that is also a plugin, changes to the following may require:

```
/reload-plugins
```

- hooks/
- .mcp.json
- agents/
- output-styles/

---

## Automatic Discovery from Parent and Nested Directories

Project Skills load from:

```
.claude/skills/
```

in:

- The starting directory.
- Parent directories up to the repository root.

This means you can start Claude Code inside a subdirectory and still inherit root-level project Skills.

When Claude works with files deeper in the repository, it can also discover nested Skills on demand.

Example:

```
packages/frontend/.claude/skills/
```

This is useful for monorepositories where individual packages need specialized Skills.

---

## Standard Skill Directory Structure

Each Skill is a directory with `SKILL.md` as the entry point.

```
my-skill/
├─ SKILL.md
├─ template.md
└─ examples/
```

```
| └─ sample.md
└─ scripts/
 └─ validate.sh
```

## Required

SKILL.md

## Optional

- Templates
- Examples
- Scripts
- Detailed reference documentation

Reference supporting files from SKILL.md so Claude knows:

- What each file contains.
- When to load it.

---

# Skills from Additional Directories

The following commands grant access to additional directories:

```
--add-dir
```

and:

```
/add-dir
```

Normally, additional directories grant file access rather than configuration discovery.

Skills are an exception.

A .claude/skills/ directory inside an added directory is loaded automatically.

This exception applies specifically to:

```
--add-dir
```

and:

```
/add-dir
```

The following setting grants file access only and does **not** automatically load Skills:

```
permissions.additionalDirectories
```

Other `.claude/` configuration from additional directories is generally not loaded automatically.

---

## Additional `CLAUDE.md` Files

`CLAUDE.md` files from additional directories are not loaded by default.

To enable them:

```
CLAUDE_CODE_ADDITIONAL_DIRECTORIES_CLAUDE_MD=1
```

See:

[Load from Additional Directories](#)

---

## Configure Skills

Skills are configured through:

1. YAML frontmatter at the top of `SKILL.md`.
  2. Markdown instructions in the body.
- 

## Types of Skill Content

### Reference Content

Reference Skills provide knowledge Claude should apply to ongoing work.



Examples:

- Conventions
- Patterns
- Style guides
- Domain knowledge

```

name: api-conventions
description: API design patterns for this codebase

```

When writing API endpoints:

- Use RESTful naming conventions.
- Return consistent error formats.
- Include request validation.

Reference content runs inline so Claude can use it alongside the current conversation.

---

## Task Content

Task Skills provide step-by-step instructions for a specific action.

Examples:

- Deployments
- Commits
- Code generation
- Release procedures

These are often better invoked manually.

Use:

```
disable-model-invocation: true
```

to prevent Claude from triggering them automatically.

Example:

```

name: deploy
```

```
description: Deploy the application to production
context: fork
disable-model-invocation: true

```

Deploy the application:

1. Run the test suite.
2. Build the application.
3. Push to the deployment target.

---

## Keep **SKILL.md** Concise

Once a Skill loads, its content remains in context across turns.

Every unnecessary line becomes a recurring context cost.

Prefer:

```
Do X.
Then do Y.
Validate Z.
```

over lengthy explanations of why the procedure exists.

For complex Skills, move detailed supporting material into separate files.

---

## Frontmatter Reference

Example:

```

name: my-skill
description: What this skill does
disable-model-invocation: true
allowed-tools: Read Grep

Your skill instructions here...
```

## Frontmatter Fields

Field	Required	Description
<code>name</code>	No	Display name shown in Skill listings. Defaults to the directory name.
<code>description</code>	Recommended	What the Skill does and when to use it. Claude uses this to decide when to apply the Skill.
<code>when_to_use</code>	No	Additional invocation context, trigger phrases, or example requests.
<code>argument-hint</code>	No	Autocomplete hint such as <code>[issue-number]</code> or <code>[filename] [format]</code> .
<code>arguments</code>	No	Named positional arguments for substitution.
<code>disable-model-invocation</code>	No	Prevents Claude from invoking the Skill automatically.
<code>user-invocable</code>	No	Controls whether the Skill appears in the <code>/</code> menu.
<code>allowed-tools</code>	No	Tools Claude may use without asking permission while the Skill is active.
<code>disallowed-tools</code>	No	Tools removed from Claude's available tool pool while the Skill is active.
<code>model</code>	No	Model override for the current turn.
<code>effort</code>	No	Effort-level override.
<code>context</code>	No	Set to <code>fork</code> to run in a forked subagent context.
<code>agent</code>	No	Subagent type used with <code>context: fork</code> .
<code>hooks</code>	No	Hooks scoped to the Skill lifecycle.
<code>paths</code>	No	Glob patterns limiting automatic activation to matching files.
<code>shell</code>	No	Shell used for dynamic shell injection.

### `name`

The display name shown in Skill listings.

Defaults to the Skill directory name.

Except for a plugin-root `SKILL.md`, this does not normally determine the command typed after `/`.

---

## description

Explains:

- What the Skill does.
- When Claude should use it.

Claude uses this field for automatic activation.

Put the most important use case first.

The combined:

- `description`
- `when_to_use`

text may be truncated in the Skill listing to reduce context usage.

---

## when\_to\_use

Provides additional context for automatic invocation.

Examples:

- Trigger phrases
- Example requests
- Specific situations

It is appended to `description`.

---

## argument-hint

Provides an autocomplete hint.

Examples:

```
[issue-number]
```

```
[filename] [format]
```

---

## arguments

Defines named positional arguments.

Example:

```
arguments:
- issue
- branch
```

Then use:

```
$issue
```

and:

```
$branch
```

inside the Skill body.

---

## disable-model-invocation

Set:

```
disable-model-invocation: true
```

to prevent Claude from loading the Skill automatically.

Use this for workflows you want to trigger manually, such as:

- `/commit`
- `/deploy`
- `/send-slack-message`

This can also prevent automatic invocation in other execution contexts.

---

## user-invocable

Set:

```
user-invocable: false
```

to hide the Skill from the  menu.

Use this for background knowledge that Claude should invoke automatically but users do not need to execute directly.

---

## allowed-tools

Allows specified tools to run without requesting additional permission while the Skill is active.

Example:

```
allowed-tools: Read Grep
```

or:

```
allowed-tools:
 - Read
 - Grep
```

This does not remove other tools.

Normal permission settings continue to govern tools not explicitly allowed here.

---

## disallowed-tools

Removes specified tools from Claude's available pool while the Skill is active.

Example use case:

- Autonomous workflows that should never ask the user a question.

The restriction clears when the user sends the next message.

---

## model

Temporarily overrides the active model for the current turn.

It accepts the same model values as:

```
/model
```

or:

```
inherit
```

The normal session model resumes on the next user prompt.

---

## effort

Overrides the current effort level.

Possible values include:

- low
- medium
- high
- xhigh
- max

Available levels depend on the selected model.

---

## context

Set:

```
context: fork
```

to execute the Skill in a forked subagent context.

---

## agent

Specifies which subagent to use with:

```
context: fork
```

Examples:

- Explore
  - Plan
  - general-purpose
  - A custom agent
- 

## hooks

Defines hooks scoped to the Skill lifecycle.

See:

[Hooks in Skills and Agents](#)

---

## paths

Limits automatic Skill activation to files matching specific glob patterns.

Uses the same format as:

[Path-Specific Rules](#)

---

## shell

Controls the shell used by dynamic context injection.

Possible values:

```
bash
```

or:



```
powershell
```

PowerShell execution on Windows requires:

```
CLAUDE_CODE_USE_POWERSHELL_TOOL=1
```

## How a Skill Gets Its Command Name

The command you type usually comes from the Skill's filesystem location.

Skill Location	Command Name Source	Example
<code>~/.claude/skills/</code> or <code>.claude/skills/</code>	Directory name	<code>.claude/skills/deploy-staging/SKILL.md</code> → <code>/deploy-staging</code>
Nested <code>.claude/skills/</code> with a naming conflict	Relative subdirectory path plus Skill directory name	<code>apps/web/.claude/skills/deploy/SKILL.md</code> → <code>/apps/web:deploy</code>
<code>.claude/commands/</code>	Filename without extension	<code>.claude/commands/deploy.md</code> → <code>/deploy</code>
Plugin <code>skills/</code> directory	Plugin namespace plus directory name	<code>my-plugin/skills/review/SKILL.md</code> → <code>/my-plugin:review</code>
Plugin-root <code>SKILL.md</code>	Frontmatter <code>name</code> , with plugin directory as fallback	<code>my-plugin/SKILL.md</code> with <code>name: review</code> → <code>/my-plugin:review</code>

The plugin-root case is the primary case where the frontmatter `name` directly determines the command name.

## Available String Substitutions

Skills support dynamic substitutions.

Variable	Description
<code>\$ARGUMENTS</code>	All arguments passed to the Skill
<code>\$ARGUMENTS[N]</code>	Argument by zero-based index
<code>\$N</code>	Shorthand for <code>\$ARGUMENTS[N]</code>

Variable	Description
<code>\$name</code>	Named argument declared in frontmatter
<code>\${CLAUDE_SESSION_ID}</code>	Current session ID
<code>\${CLAUDE_EFFORT}</code>	Current effort level
<code>\${CLAUDE_SKILL_DIR}</code>	Directory containing the Skill's <code>SKILL.md</code>
<code>\${CLAUDE_PROJECT_DIR}</code>	Project root directory

## `$ARGUMENTS`

Contains all arguments passed after the Skill name.

Example:

```
/fix-issue 123
```

with:

```
$ARGUMENTS
```

becomes:

```
123
```

If the Skill does not explicitly contain `$ARGUMENTS`, Claude Code can append:

```
ARGUMENTS: <value>
```

to the rendered Skill content.

## Indexed Arguments

Use:

```
$ARGUMENTS[0]
```

or the shorter:

```
$0
```

Example:

```
/migrate-component SearchBar React Vue
```

Then:

```
$0
```

becomes:

```
SearchBar
```

```
$1
```

becomes:

```
React
```

```
$2
```

becomes:

```
Vue
```

---

## Named Arguments

Frontmatter:

```
arguments:
- issue
- branch
```

Skill body:

```
Fix issue $issue on branch $branch.
```

Named arguments map to positional arguments in order.

---

## Session ID

Use:

```
${CLAUDE_SESSION_ID}
```

for:

- Logging
- Session-specific files
- Correlating outputs

Example:

```

name: session-logger
description: Log activity for this session

Log the following to:

logs/${CLAUDE_SESSION_ID}.log

$ARGUMENTS
```

---

## Current Effort

Use:

```
${CLAUDE_EFFORT}
```

Possible values include:

- low
- medium
- high
- xhigh
- max

This lets a Skill adapt its behavior to the active reasoning effort.

---

## Skill Directory

Use:

```
${CLAUDE_SKILL_DIR}
```

to reference files bundled with the Skill regardless of where the Skill is installed.

Example:

```
python3 ${CLAUDE_SKILL_DIR}/scripts/helper.py
```

---

## Project Directory

Use:

```
${CLAUDE_PROJECT_DIR}
```

to reference the project root.

Example:

```
${CLAUDE_PROJECT_DIR}/scripts/lint.sh
```

---

## Quoting Arguments

Indexed arguments use shell-style quoting.

Example:

```
/my-skill "hello world" second
```

Then:

```
$0
```

becomes:

```
hello world
```

and:

```
$1
```

becomes:

```
second
```

---

## Escaping Literal Dollar Signs

To include a literal dollar sign before a substitution-like token, escape it.

Example:

```
\$1.00
```

---

## Add Supporting Files

A Skill can contain multiple files.

```
my-skill/
├─ SKILL.md
├─ reference.md
├─ examples.md
└─ scripts/
 └─ helper.py
```

Reference these files from `SKILL.md`.

Example:

```
Additional Resources

- For complete API details, see reference.md.
- For usage examples, see examples.md.
```

This keeps `SKILL.md` focused while allowing Claude to load detailed material only when necessary.

### Tip

Keep `SKILL.md` under approximately 500 lines.

Move detailed reference material into supporting files.

---

## Control Who Invokes a Skill

By default:

- You can invoke a Skill.
- Claude can invoke the Skill automatically.

Two frontmatter options change this behavior.

---

### Manual Invocation Only

Use:

```
disable-model-invocation: true
```

Example:

```

name: deploy
description: Deploy the application to production
disable-model-invocation: true

```

Deploy \$ARGUMENTS to production:

1. Run the test suite.
2. Build the application.
3. Push to the deployment target.
4. Verify the deployment succeeded.

This is useful for workflows with side effects.

You probably do not want Claude deciding, "The code looks good. Surprise! I deployed production."

## Claude-Only Invocation

Use:

```
user-invocable: false
```

This hides the Skill from the  menu.

Use it for background knowledge such as:

```
legacy-system-context
```

Claude should use the information automatically when relevant, but users do not need a command for it.

## Invocation Matrix

Frontmatter	You Can Invoke	Claude Can Invoke	Context Behavior
Default	Yes	Yes	Description is listed; full Skill loads when invoked
<code>disable-model-invocation: true</code>	Yes	No	Hidden from Claude's automatic Skill list



Frontmatter	You Can Invoke	Claude Can Invoke	Context Behavior
<code>user-invocable: false</code>	No	Yes	Description remains available to Claude

## Skill Content Lifecycle

When a Skill is invoked, its rendered `SKILL.md` content enters the conversation as a single message.

That content remains in the session context.

Claude Code does not automatically reread the Skill file on every later turn.

Write guidance that should continue to apply as persistent instructions.

## Reinvocation

If Claude invokes a Skill again and the rendered content is identical to the version already in context, Claude Code may add only a short note indicating that the Skill is already loaded.

If the rendered content changes because:

- Arguments changed.
- Dynamic context produced different output.

then the updated full content can be appended again.

## Auto-Compaction

When the conversation is compacted to free context, Claude Code can carry invoked Skills forward within a token budget.

More recently invoked Skills receive priority.

Older Skills may eventually be dropped if many large Skills have been loaded.

If a Skill seems to stop influencing Claude after compaction:

- Reinvoke it.
- Strengthen its instructions.

- Reduce unnecessary Skill size.
- Use hooks when deterministic enforcement is required.

---

## Pre-Approve Tools for a Skill

The `allowed-tools` field lets specified tools run without per-use approval while the Skill is active.

Example:

```

name: commit
description: Stage and commit the current changes
disable-model-invocation: true
allowed-tools: Bash(git add *) Bash(git commit *) Bash(git status *)

```

This allows relevant Git commands without repeated approval prompts.

It does not remove other tools.

Normal permission settings still apply to tools not explicitly pre-approved.

---

## Project Trust

For Skills checked into:

```
.claude/skills/
```

the `allowed-tools` configuration takes effect after you accept the workspace trust dialog.

Review project Skills before trusting a repository.

A Skill can potentially grant itself broad tool access.

---

## Remove Tools Temporarily

Use:

```
disallowed-tools:
```

to remove tools from Claude's available pool while the Skill is active.

The restriction clears when you send the next message.

For permanent restrictions, configure your normal permission deny rules.

---

## Pass Arguments to Skills

Example:

```

name: fix-issue
description: Fix a GitHub issue
disable-model-invocation: true

```

Fix GitHub issue \$ARGUMENTS following our coding standards.

1. Read the issue description.
2. Understand the requirements.
3. Implement the fix.
4. Write tests.
5. Create a commit.

Run:

```
/fix-issue 123
```

Claude receives:

```
Fix GitHub issue 123 following our coding standards.
```

---

## Stack Multiple Skills

You can invoke multiple inline Skills in one message.

Example:

```
/code-review /fix-issue 123
```

Claude Code can load multiple Skills and pass the trailing arguments to them.

Expansion stops when it reaches content that is not another compatible inline user-invocable Skill.

---

## Individual Positional Arguments

Example:

```

name: migrate-component
description: Migrate a component from one framework to another

Migrate the $ARGUMENTS[0] component from $ARGUMENTS[1] to $ARGUMENTS[2].

Preserve all existing behavior and tests.
```

Run:

```
/migrate-component SearchBar React Vue
```

Equivalent shorthand:

```

name: migrate-component
description: Migrate a component from one framework to another

Migrate the $0 component from $1 to $2.

Preserve all existing behavior and tests.
```

---

# Advanced Patterns

## Inject Dynamic Context

The syntax:

```
!`<command>`
```

runs a shell command before the Skill content is sent to Claude.

The command output replaces the placeholder.

Claude sees the actual output, not the original command.

---

## Pull Request Summary Example

```

name: pr-summary
description: Summarize changes in a pull request
context: fork
agent: Explore
allowed-tools: Bash(gh *)

Pull Request Context

- PR diff: !`gh pr diff`
- PR comments: !`gh pr view --comments`
- Changed files: !`gh pr diff --name-only`

Your Task

Summarize this pull request.
```

When the Skill runs:

1. Each shell command executes.
2. The output replaces the placeholder.
3. Claude receives the fully rendered prompt containing the live PR data.

This is preprocessing.

Claude sees only the final rendered content.

---

## No Recursive Expansion

Command output is inserted as plain text.

It is not rescanned for additional dynamic command placeholders.

A command therefore cannot emit another placeholder and trigger a second expansion pass.

---

## Inline Syntax Rules

The inline command syntax is recognized when `!` appears:

- At the start of a line.
- Immediately after whitespace.

Example that works:

```
Output: !`git status`
```

Example that remains literal:

```
KEY=!`cmd`
```

---

## Multi-Line Dynamic Commands

Use a fenced block opened with:

```
```\nnode --version\nnpm --version\ngit status --short\n```\n
```

Example:

```
## Environment

```!
node --version
npm --version
git status --short
```
```

Disable Skill Shell Execution

Set:

```
{
  "disableSkillShellExecution": true
}
```

to prevent shell execution from:

- User Skills
- Project Skills
- Plugin Skills
- Additional-directory Skills
- Custom commands

The command is replaced with:

```
[shell command execution disabled by policy]
```

Bundled and managed Skills are not affected.

This is especially useful in managed enterprise settings.

Request Deeper Reasoning

Include:

```
ultrathink
```

anywhere in the Skill content to request deeper reasoning when the Skill runs.

Run Skills in a Subagent

Add:

```
context: fork
```

to run a Skill in an isolated subagent context.

The Skill content becomes the prompt driving the subagent.

The subagent does not receive the full conversation history.

Warning

`context: fork` makes sense for Skills with an explicit task.

A Skill containing only guidelines such as:

```
Use these API conventions.
```

gives the subagent no meaningful task to perform.

Skills and Subagents

Skills and subagents can work together in two directions.

| Approach | System Prompt | Task | Also Loads |
|---|------------------------|-------------------------------|--|
| Skill with <code>context: fork</code> | From agent type | <code>SKILL.md</code> content | <code>CLAUDE.md</code> , except for some built-in lightweight agents |
| Subagent with <code>skills</code> field | Subagent Markdown body | Claude's delegation message | Preloaded Skills and <code>CLAUDE.md</code> |

With:

```
context: fork
```


you:

1. Define the task in the Skill.
2. Select an agent type.
3. Run the task in an isolated context.

Example: Research Skill Using the Explore Agent

```
---
name: deep-research
description: Research a topic thoroughly
context: fork
agent: Explore
---

Research $ARGUMENTS thoroughly:

1. Find relevant files using Glob and Grep.
2. Read and analyze the code.
3. Summarize findings with specific file references.
```

When the Skill runs:

1. Claude Code creates an isolated context.
2. The subagent receives the Skill content.
3. The `agent` field determines the environment.
4. The results are summarized and returned to the main conversation.

Possible agent values include:

- `Explore`
- `Plan`
- `general-purpose`
- Custom agents

If omitted, the default is typically:

```
general-purpose
```

Restrict Claude's Skill Access

By default, Claude can invoke any Skill that does not contain:

```
disable-model-invocation: true
```

There are three primary ways to control access.

Disable All Skills

Deny the Skill tool in:

```
/permissions
```

Example deny rule:

```
Skill
```

Allow or Deny Specific Skills

Examples:

```
Skill(commit)
```

```
Skill(review-pr *)
```

```
Skill(deploy *)
```

Syntax:

```
Skill(name)
```

for exact matches.

Use:

```
Skill(name *)
```

to match the Skill plus arguments.

Hide Individual Skills

Add:

```
disable-model-invocation: true
```

to the Skill frontmatter.

This removes the Skill from Claude's automatic Skill context.

Note

`user-invocable` controls menu visibility.

It does not block programmatic Skill invocation.

Use:

```
disable-model-invocation: true
```

to block automatic invocation.

Override Skill Visibility from Settings

The:

```
skillOverrides
```

setting controls Skill visibility without editing the Skill's own `SKILL.md`.

This is useful for Skills:

- Checked into shared repositories.
- Provided externally.

- Managed outside your control.

The `/skills` menu can manage these settings.

Override States

| Value | Listed to Claude | Shown in <code>/</code> Menu |
|------------------------------------|----------------------|------------------------------|
| <code>"on"</code> | Name and description | Yes |
| <code>"name-only"</code> | Name only | Yes |
| <code>"user-invocable-only"</code> | Hidden | Yes |
| <code>"off"</code> | Hidden | Hidden |

A Skill not listed in `skill0overrides` is treated as:

on

Example:

```
{
  "skill0overrides": {
    "legacy-context": "name-only",
    "deploy": "off"
  }
}
```

Plugin Skills are managed through:

`/plugin`

rather than `skill0overrides`.

Evaluate and Iterate on a Skill

A Skill triggering successfully does not necessarily mean it performs well.

Evaluate two separate things:

1. **Triggering:** Does Claude invoke the Skill when it should?
2. **Output quality:** Does the Skill produce the expected result?

The best evaluation method is a baseline comparison.

For realistic prompts:

1. Run each prompt in a fresh session with the Skill enabled.
2. Run the same prompt with the Skill disabled.
3. Compare the results.

Fresh sessions matter because previous context can hide weaknesses in the Skill instructions.

Run Evals with `skill-creator`

The official `skill-creator` plugin automates Skill evaluation.

Install it:

```
/plugin install skill-creator@claude-plugins-official
```

If the plugin marketplace is missing or outdated:

```
/plugin marketplace update claude-plugins-official
```

or:

```
/plugin marketplace add anthropics/claude-plugins-official
```

Then reload plugins:

```
/reload-plugins
```

Ask Claude:

```
Evaluate my summarize-changes skill with skill-creator.
```

What `skill-creator` Can Do

Test Cases

Stores:

- Prompts
- Input files
- Expected behavior

in:

```
evals/evals.json
```

inside the Skill directory.

Isolated Runs

Runs each test case in a fresh subagent context.

It can record:

- Token count
- Execution duration

Grading

Checks assertions and writes evidence to:

```
grading.json
```

Benchmarking

Aggregates:

- Pass rate
- Time
- Token use

for comparisons between:

- With-Skill runs
- Without-Skill runs

Results can be stored in:

```
benchmark.json
```

Version Comparison

Runs blind A/B comparisons between Skill versions.

This helps confirm whether an edit is actually an improvement.

Description Tuning

Generates:

- Should-trigger prompts
- Should-not-trigger prompts

Then measures activation accuracy and proposes improved descriptions.

Review Viewer

Opens an HTML report for inspecting outputs and recording qualitative feedback.

For the evaluation format and workflow:

[Evaluating Skill Output Quality](#)

For additional background:

[Improving Skill Creator: Test, Measure, and Refine Agent Skills](#)

Share Skills

Skills can be distributed at different scopes.

Project Skills

Commit:

```
.claude/skills/
```

to version control.

Plugins

Create:

```
skills/
```

inside a plugin.

Managed Skills

Deploy organization-wide through managed settings.

Generate Visual Output

Skills can bundle and execute scripts in any language.

This makes them useful for generating visual artifacts such as:

- Interactive HTML reports
 - Dependency graphs
 - Test coverage reports
 - API documentation
 - Database schema visualizations
 - Codebase explorers
-

Example: Codebase Visualizer

Create the Skill directory:

```
mkdir -p ~/.claude/skills/codebase-visualizer/scripts
```

Save the following as:

```
~/.claude/skills/codebase-visualizer/SKILL.md
```

```
---  
name: codebase-visualizer  
description: Generate an interactive collapsible tree visualization of your
```



```

codebase. Use when exploring a new repo, understanding project structure, or
identifying large files.
allowed-tools: Bash(python3 *)
---

# Codebase Visualizer

Generate an interactive HTML tree view that shows your project's file structure
with collapsible directories.

## Usage

Run the visualization script from your project root:

```bash
python3 ${CLAUDE_SKILL_DIR}/scripts/visualize.py .
```

This creates:

```text
codebase-map.html
```

in the current directory and opens it in your default browser.

## What the Visualization Shows

- Collapsible directories: Click folders to expand or collapse them.
- File sizes: Displayed next to each file.
- Colors: Different colors for different file types.
- Directory totals: Aggregate size for each folder.

```

Save the script as:

```
~/.claude/skills/codebase-visualizer/scripts/visualize.py
```

```

#!/usr/bin/env python3
"""Generate an interactive collapsible tree visualization of a codebase."""

import json
import sys
import webbrowser
from collections import Counter
from html import escape

```

```

from pathlib import Path

IGNORE = {
    ".git",
    "node_modules",
    "__pycache__",
    ".venv",
    "venv",
    "dist",
    "build",
}

def scan(path: Path, stats: dict) -> dict:
    result = {
        "name": path.name,
        "children": [],
        "size": 0,
    }

    try:
        for item in sorted(path.iterdir()):
            if item.name in IGNORE or item.name.startswith("."):
                continue

            if item.is_file():
                size = item.stat().st_size
                ext = item.suffix.lower() or "(no ext)"

                result["children"].append(
                    {
                        "name": item.name,
                        "size": size,
                        "ext": ext,
                    }
                )

                result["size"] += size
                stats["files"] += 1
                stats["extensions"][ext] += 1
                stats["ext_sizes"][ext] += size

            elif item.is_dir():
                stats["dirs"] += 1
                child = scan(item, stats)

                if child["children"]:
                    result["children"].append(child)

```

```

        result["size"] += child["size"]

except PermissionError:
    pass

return result

def generate_html(data: dict, stats: dict, output: Path) -> None:
    ext_sizes = stats["ext_sizes"]
    total_size = sum(ext_sizes.values()) or 1
    sorted_exts = sorted(
        ext_sizes.items(),
        key=lambda x: -x[1],
   )[:8]

    colors = {
        ".js": "#f7df1e",
        ".ts": "#3178c6",
        ".py": "#3776ab",
        ".go": "#00add8",
        ".rs": "#dea584",
        ".rb": "#cc342d",
        ".css": "#264de4",
        ".html": "#e34c26",
        ".json": "#6b7280",
        ".md": "#083fa1",
        ".yaml": "#cb171e",
        ".yml": "#cb171e",
        ".mdx": "#083fa1",
        ".tsx": "#3178c6",
        ".jsx": "#61dafb",
        ".sh": "#4eaa25",
    }

    langBars = "".join(
        f'<div class="bar-row">'
        f'<span class="bar-label">{ext}</span>'
        f'<div class="bar" '
        f'style="width: {(size / total_size) * 100}%;'
        f'background: {colors.get(ext, "#6b7280")}"></div>'
        f'<span class="bar-pct">'
        f' {(size / total_size) * 100:.1f}% '
        f'</span>'
        f'</div>'
        for ext, size in sorted_exts
    )

```

```

def fmt(b):
    if b < 1024:
        return f"{b} B"

    if b < 1048576:
        return f"{b / 1024:.1f} KB"

    return f"{b / 1048576:.1f} MB"

html = f"""<!DOCTYPE html>
<html>
<head>
<meta charset="utf-8">
<title>Codebase Explorer</title>

<style>
body {{
    font: 14px/1.5 system-ui, sans-serif;
    margin: 0;
    background: #1a1a2e;
    color: #eee;
}}

.container {{
    display: flex;
    height: 100vh;
}}

.sidebar {{
    width: 280px;
    background: #252542;
    padding: 20px;
    border-right: 1px solid #3d3d5c;
    overflow-y: auto;
    flex-shrink: 0;
}}

.main {{
    flex: 1;
    padding: 20px;
    overflow-y: auto;
}}

h1 {{
    margin: 0 0 10px 0;
    font-size: 18px;
}}

```

```
h2 {{
  margin: 20px 0 10px 0;
  font-size: 14px;
  color: #888;
  text-transform: uppercase;
}}

.stat {{
  display: flex;
  justify-content: space-between;
  padding: 8px 0;
  border-bottom: 1px solid #3d3d5c;
}}

.stat-value {{
  font-weight: bold;
}}

.bar-row {{
  display: flex;
  align-items: center;
  margin: 6px 0;
}}

.bar-label {{
  width: 55px;
  font-size: 12px;
  color: #aaa;
}}

.bar {{
  height: 18px;
  border-radius: 3px;
}}

.bar-pct {{
  margin-left: 8px;
  font-size: 12px;
  color: #666;
}}

.tree {{
  list-style: none;
  padding-left: 20px;
}}

details {{
  cursor: pointer;
```

```

    }}

    summary {{
        padding: 4px 8px;
        border-radius: 4px;
    }}

    summary:hover {{
        background: #2d2d44;
    }}

    .folder {{
        color: #ffd700;
    }}

    .file {{
        display: flex;
        align-items: center;
        padding: 4px 8px;
        border-radius: 4px;
    }}

    .file:hover {{
        background: #2d2d44;
    }}

    .size {{
        color: #888;
        margin-left: auto;
        font-size: 12px;
    }}

    .dot {{
        width: 8px;
        height: 8px;
        border-radius: 50%;
        margin-right: 8px;
    }}
</style>
</head>

<body>
    <div class="container">
        <div class="sidebar">
            <h1><img alt="Summary icon" data-bbox="232 842 252 857" /> Summary</h1>

            <div class="stat">
                <span>Files</span>
            </div>
        </div>
    </div>

```

```

    <span class="stat-value">{stats["files"]:,}</span>
</div>

<div class="stat">
    <span>Directories</span>
    <span class="stat-value">{stats["dirs"]:,}</span>
</div>

<div class="stat">
    <span>Total size</span>
    <span class="stat-value">{fmt(data["size"])}</span>
</div>

<div class="stat">
    <span>File types</span>
    <span class="stat-value">{len(stats["extensions"])}</span>
</div>

<h2>By File Type</h2>

    {lang_bars}
</div>

<div class="main">
    <h1>📁 {escape(data["name"])}</h1>
    <ul class="tree" id="root"></ul>
</div>
</div>

<script>
    const data = {json.dumps(data)};
    const colors = {json.dumps(colors)};

    function fmt(b) {{
        if (b < 1024) return b + " B";
        if (b < 1048576) return (b / 1024).toFixed(1) + " KB";
        return (b / 1048576).toFixed(1) + " MB";
    }}

    function esc(s) {{
        return s.replace(
            /[\<>'"]/g,
            c => ({
                "&": "&amp;",
                "<": "&lt;",
                ">": "&gt;",
                "'": "&quot;",
                '"': "&#39;"
            })
        )
    }}

```

```

    }}[c])
  );
}}

function render(node, parent) {{
  if (node.children) {{
    const det = document.createElement("details");

    det.open = parent === document.getElementById("root");

    det.innerHTML =
      `<summary>` +
      `<span class="folder">📁 ${esc(node.name)}</span>` +
      `<span class="size">${fmt(node.size)}</span>` +
      `</summary>`;

    const ul = document.createElement("ul");
    ul.className = "tree";

    node.children.sort(
      (a, b) =>
        (b.children ? 1 : 0) -
        (a.children ? 1 : 0) ||
        a.name.localeCompare(b.name)
    );

    node.children.forEach(c => render(c, ul));

    det.appendChild(ul);

    const li = document.createElement("li");
    li.appendChild(det);
    parent.appendChild(li);

  }} else {{
    const li = document.createElement("li");
    li.className = "file";

    li.innerHTML =
      `<span class="dot" ` +
      `style="background:${colors[node.ext] || "#6b7280"}">` +
      `</span>` +
      `${esc(node.name)}` +
      `<span class="size">${fmt(node.size)}</span>`;

    parent.appendChild(li);
  }}
}}

```



```

        data.children.forEach(
            c => render(c, document.getElementById("root"))
        );
    </script>
</body>
</html>
"""

    output.write_text(html)

if __name__ == "__main__":
    target = Path(
        sys.argv[1] if len(sys.argv) > 1 else "."
    ).resolve()

    stats = {
        "files": 0,
        "dirs": 0,
        "extensions": Counter(),
        "ext_sizes": Counter(),
    }

    data = scan(target, stats)

    out = Path("codebase-map.html")

    generate_html(
        data,
        stats,
        out,
    )

    print(f"Generated {out.absolute()}")

    webbrowser.open(
        f"file://{out.absolute()}"
    )

```

To test the Skill, open Claude Code in any project and ask:

Visualize this codebase.

Claude runs the script, generates:

```
codebase-map.html
```

and opens the result in your browser.

This pattern also works for:

- Dependency graphs
- Test coverage reports
- API documentation
- Database schema visualizations
- Interactive debugging tools
- Data-analysis reports

The bundled script performs the deterministic work while Claude handles orchestration.

Troubleshooting

Skill Not Triggering

If Claude does not use your Skill when expected:

1. Check that the description includes phrases users would naturally say.
2. Verify the Skill appears when you ask:

```
What skills are available?
```

1. Rephrase the request so it better matches the Skill description.
2. Invoke the Skill directly:

```
/skill-name
```

If the YAML frontmatter is malformed, Claude Code may still load the Skill body but with empty metadata.

In that case:

- `/skill-name` may still work.
- Automatic activation may fail because Claude has no valid `description`.

Run Claude Code with:

```
--debug
```

to inspect parsing errors.

Skill Triggers Too Often

If Claude invokes a Skill when it should not:

1. Make the description more specific.
2. Add:

```
disable-model-invocation: true
```

if you only want manual invocation.

Skill Descriptions Are Cut Short

Claude Code loads a listing of available Skill names and descriptions so Claude knows what Skills exist.

When many Skills are installed, the listing may exceed its context budget.

Claude Code can shorten or drop descriptions to fit.

The listing still contains Skill names, but some descriptions may be truncated.

This can remove important matching keywords and reduce automatic activation accuracy.

Diagnose Skill Listing Size

Run:

```
/doctor
```

for an estimate of:

- Skill-listing context cost
- Largest contributors

The Skills row in:

```
/context
```

shows the size of the Skill listing after budget rules are applied.

Increase the Skill Listing Budget

Configure:

```
skillListingBudgetFraction
```

Example:

```
{
  "skillListingBudgetFraction": 0.02
}
```

This allocates approximately 2% of the model's context window to Skill listings.

You can also use:

```
SLASH_COMMAND_TOOL_CHAR_BUDGET
```

to set a fixed character budget.

Reduce Skill Listing Cost

For lower-priority Skills, use:

```
{
  "skillOverrides": {
    "low-priority-skill": "name-only"
  }
}
```

This keeps the Skill name visible while removing the description from the listing.

You can also shorten:

- `description`
- `when_to_use`

Put the most important trigger information first.

Related Resources

- [Debug Your Configuration](#)
- [Evaluating Skill Output Quality](#)
- [Skill Authoring Best Practices](#)
- [Subagents](#)
- [Plugins](#)
- [Hooks](#)
- [Memory](#)
- [Commands](#)
- [Permissions](#)
- [Claude Tag Skills](#)